

Lab 5

Programing with Python Lab

BCACC393

1. Non-Parameterized Function

Write a Python program using a **non-parameterized function** `greet()` that displays:

Welcome to Python Programming

Call the function from the main program.

2. Function with No Arguments

Write a function `display_info()` that takes **no arguments** and displays your name, department, and college name.

3. Function with Positional Arguments

Write a function `add_numbers(a, b)` that accepts two numbers as **positional arguments** and displays their sum, difference, product, and quotient.

4. Parameterized Function

Write a function `calculate_square(n)` that accepts a number as a parameter and displays its square and cube.

5. Void Function

Write a **void function** `display_table(n)` that accepts a number and prints its multiplication table from 1 to 10. The function should not return any value.

6. Fruitful Function

Write a **fruitful function** `calculate_area(length, width)` that calculates and **returns** the area of a rectangle. Display the returned value outside the function.

7. Function Returning Multiple Values

Write a function `calculate(a, b)` that accepts two numbers and returns their sum, difference, product, and quotient. Store the returned values in separate variables and display them.

8. Function with Positional Arguments

Write a function `student_result(name, marks1, marks2, marks3)` that accepts student information using **positional arguments** and calculates the total marks and percentage.

9. Function with Keyword Arguments

Write a function `student_details(name, age, course)` and call it using **keyword arguments** in a different order from the parameter declaration.

10. Positional and Keyword Arguments

Write a function `employee_details(name, department, salary)` and demonstrate calling it using:

1. Only positional arguments
2. Only keyword arguments
3. A combination of positional and keyword arguments

11. Default Arguments

Write a function `power(base, exponent=2)` that calculates the power of a number. Demonstrate the function with:

- Only the base
- Both base and exponent

12. Function to Check Even or Odd

Write a **fruitful function** `is_even(n)` that returns `True` if the given number is even and `False` otherwise. Use the returned value to display an appropriate message.

13. Lambda Function — Square

Write a Python program using a **lambda function** to calculate the square of a number.

14. Lambda Function — Maximum

Write a lambda function that accepts two numbers and returns the **larger number**.

15. Lambda Function — Conditional Expression

Write a lambda function that accepts a number and returns "Positive", "Negative", or "Zero" depending on its value.

16. `map()` Function

Given the list:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Use the `map()` function and a lambda expression to create a new list containing the **squares** of all numbers.

17. `map()` with Multiple Lists

Given:

```
a = [10, 20, 30, 40, 50]
b = [1, 2, 3, 4, 5]
```

Use `map()` to create a new list containing the **sum of corresponding elements** of the two lists.

Expected result:

```
[11, 22, 33, 44, 55]
```

18. `filter()` Function

Given:

```
numbers = [10, 15, 20, 25, 30, 35, 40, 45, 50]
```

Use `filter()` and a lambda function to create a list containing only the **even numbers**.

19. `filter()` — Prime Numbers

Write a Python program using `filter()` to extract all **prime numbers** from:

```
numbers = [2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 13, 15, 17, 20]
```

20. `reduce()` Function

Use the `reduce()` function from the `functools` module to calculate the **sum of all elements** in:

```
numbers = [10, 20, 30, 40, 50]
```

21. `reduce()` — Product

Use `reduce()` and a lambda function to calculate the **product of all numbers** from 1 to 10.

22. Iterators

Create a Python list containing five numbers. Convert the list into an **iterator** using `iter()`. Use `next()` to display each element one by one.

Also demonstrate what happens when `next()` is called after all elements have been consumed.

23. Custom Iterator

Create a custom iterator class `CountDown` that accepts a starting number and returns numbers in descending order until it reaches 1.

For example:

```
CountDown(5)
5
4
3
2
1
```

24. Generator Function

Write a **generator function** `generate_numbers(n)` that generates numbers from 1 to `n` using the `yield` statement.

Use a `for` loop to display the generated values.

25. Generator + Filtering

Write a generator function that generates all numbers from 1 to 50. Use another function or suitable logic to display only the numbers that are divisible by both **3 and 5**.